

Tydzień 11; grupa średniozaawansowana

13.05.2025

Zadanie Bielany

W pliku tekstowym `warszawa_bielany.txt` znajdują się dzienne pomiary temperatury maksymalnej zarejestrowane przez stację meteorologiczną w Warszawie-Bielanach w latach od 1951 do 2019.

Każda linia pliku ma następujący format:

```
rok miesiąc dzień największa_temperatura_dobowa
```

Przykład:

```
1951 07 01 29.5
```

Napisz program, który:

- otworzy i odczyta zawartość pliku,
- zliczy, ile dni upalnych (czyli takich, w których temperatura przekraczała 30°C) wystąpiło w każdym roku,
- wypisze na ekran liczbę dni upalnych w kolejnych latach,
- do przechowywania wyników użyje kontenera `std::map`.

Uwagi:

- Przy odczycie danych sprawdź, czy plik otworzył się poprawnie.
- Do przechowywania wyników wykorzystaj mapę, gdzie kluczem będzie rok, a wartością liczba dni upalnych.
- Wypisz wyniki zarówno z użyciem iteratora, jak i pętli zasięgu (`range-based for`).

Zadanie analiza

Napisz program, który wczytuje dane z pliku tekstowego przekazanego za pomocą przekierowania standardowego wejścia (np. `./a.out < dane.txt`).

Zakłada się, że każda linia w pliku ma następujący format:

Imię Nazwisko liczba

gdzie:

- Imię i Nazwisko to ciągi znaków (zmiennie typu `std::string`),
- liczba to wartość typu `float` (np. ocena, wynik, temperatura itp.).

Program powinien wykonać następujące kroki:

1. **Wczytać dane** z pliku, ignorując pierwsze dwie kolumny (imię i nazwisko) i zapisać tylko liczby z trzeciej kolumny do wektora typu `std::vector<float>`.
2. **Po wczytaniu wszystkich danych**, program powinien:
 - Obliczyć **średnią arytmetyczną** wszystkich wczytanych liczb.
 - Wyświetlić **najmniejszą i największą** liczbę.
 - Wyświetlić **liczbę rekordów** (czyli liczb wczytanych danych).
3. **Wszystkie wczytane liczby** powinny być wypisane na ekran w odpowiednim formacie z dwoma miejscami po przecinku.
4. Program powinien obsługiwać sytuacje, w których:
 - Plik zawiera błędne dane (np. brak liczby w trzeciej kolumnie).
 - Plik jest pusty.

Sposób uruchamiania programu:

```
./a.out < dane.txt
```

Przykładowa zawartość pliku dane.txt:

```
Anna Kowalska 78.5
Jan Nowak 92.0
Zofia Malinowska 65.5
```

Przykład wyników działania programu:

```
Wczytane liczby:
78.50
92.00
65.50
```

```
Statystyki:
Liczba rekordów: 3
Średnia: 78.67
Min: 65.5, Max: 92
```

Uwagi

- Program korzysta z `getline` do wczytania całej linii, a następnie używa `istreamstream` do wydzielenia trzeciej kolumny (liczby) z każdej linii.
- W przypadku błędnego formatu linii program wypisuje komunikat o błędzie.

Zadanie Losowanie liczb i zapis do plików

Napisz program, który:

1. Wczytuje od użytkownika liczbę całkowitą n oznaczającą ile liczb należy wylosować.
2. Losuje n liczb całkowitych z przedziału od 1 do 100.
3. Dla każdej wylosowanej liczby sprawdza, czy jest parzysta czy nieparzysta.
4. Liczby parzyste zapisuje do pliku tekstowego o nazwie `parzyste.txt`, a liczby nieparzyste do pliku `nieparzyste.txt`.
5. Program powinien używać biblioteki `fstream`, a do zapisu danych do plików należy wykorzystać klasę `std::ofstream`.
6. Program powinien sprawdzać, czy pliki zostały poprawnie otwarte. W przypadku błędu otwarcia pliku należy wyświetlić odpowiedni komunikat.

Uwagi:

- Do losowania liczb można użyć funkcji `rand()` z biblioteki `cstdlib`, lub nowoczesnego generatora liczb pseudolosowych z biblioteki `<random>`.
- Każda liczba powinna być zapisana w osobnej linii odpowiedniego pliku.
- W programie nie należy używać przekierowania wejścia/wyjścia w terminalu – pliki powinny być otwierane i obsługiwane w kodzie źródłowym.

Zadanie analiza2

Napisz program, który:

1. Wczytuje dane z pliku tekstowego zawierającego dane w następującym formacie:

```
nazwa_kategorii  wartość
```

Gdzie `nazwa_kategorii` to ciąg znaków (typ `std::string`), a `wartość` to liczba zmiennoprzecinkowa (typ `float`).

2. Program powinien wczytać wszystkie dane i przechować je w pojemniku typu `std::map<std::string, float>`. Kluczami w mapie będą kategorie (`nazwa_kategorii`), a wartościami odpowiadające im liczby.
3. Po wczytaniu danych, program powinien:
 - Obliczyć sumę wartości dla każdej kategorii,
 - Obliczyć średnią wartość dla każdej kategorii,
 - Wypisać na ekran wszystkie wczytane dane, a także obliczoną sumę oraz średnią dla każdej kategorii.
4. Program powinien obsługiwać przypadki, gdy plik jest pusty lub zawiera błędne dane (np. brak wartości liczbowej).
5. Program powinien być uruchamiany w sposób opisany poniżej.

Przykład

Zawartość pliku `kategorie.txt`:

```
A 12.5
B 15.7
A 10.4
C 8.3
B 9.1
C 7.7
```

Wynik działania programu:

Dane:

```
A: 12.5, 10.4
B: 15.7, 9.1
C: 8.3, 7.7
```

```
Suma dla kategorii A: 22.9
Średnia dla kategorii A: 11.45
```

```
Suma dla kategorii B: 24.8
Średnia dla kategorii B: 12.4
```

```
Suma dla kategorii C: 16.0
Średnia dla kategorii C: 8.0
```

Wymagania

- Program powinien korzystać z pojemnika `std::map` do przechowywania kategorii i odpowiadających im wartości.
- Program powinien obsługiwać przypadki, gdy plik jest pusty lub zawiera błędne dane.
- Program powinien używać odpowiednich mechanizmów wczytywania danych z pliku, np. `ifstream`.
- Program powinien wypisać wszystkie kategorie, sumy i średnie dla każdej kategorii.